Testing, CI/CD and Operations

Pipeline, quality gates, environments and runbooks

1. Pipeline stages (GitHub Actions)

Stage	What runs	Blocks merge if
Lint and types	ruff check, ruff format, mypy	Any error
Unit tests	pytest tests/unit with coverage	Coverage under 80% or any failure
Detection-quality gate	Replays labelled attack and benign scenarios	Recall under 0.90, FPR over 0.05, or detection later than 3 events
Integration tests	Gateway + detector + Postgres service	Any failure
Security	bandit, pip-audit, gitleaks secret scan	Medium+ code issue, known vuln, or leaked secret
Build image	Docker build + Trivy scan	Critical or high image vulnerability
Release (on tag vX.Y.Z)	Push image, SBOM, release notes	Any failure
Deploy staging then prod	Canary 10%, smoke test, manual approval for prod	Smoke test or canary health fails; auto rollback

The detection-quality gate is the key addition: the build fails when the detector gets worse, not only when code breaks. Workflow files are in `.github/workflows/ci.yml` and `release.yml`.

2. Branching and releases

- Trunk-based: short-lived feature branches, pull request into main, squash merge. Main is always deployable.
- Semantic version tags (v1.2.0) trigger a release. Manifest changes are versioned separately and need owner sign-off.
- New detection rules and models ship in **shadow mode** first (score and log only), then are enabled after review of their false-positive rate.

3. Environments

Environment	Purpose	Data
Local / CI	Development and automated tests	Synthetic scenarios only
Staging	Canary, smoke test, load test, demo rehearsal	Synthetic + replayed anonymised traffic
Production	Live monitoring of real agents	Real events; PII masked in stored payloads

4. Test suite

Type	Examples
Unit	Rule engine, taint propagation, scoring maths, manifest signature check, authority capping
Scenario (labelled)	Procurement PDF injection, stolen token, new-tool legit change, slow drift, mimicry of normal frequencies, honeypot access, agent-chain compromise, benign task change
Integration	Gateway to detector to response controller; approval webhook; token restriction
Load	Target: 1,000 events/s per gateway node with inline p95 under 10 ms
Security	Log tampering, baseline poisoning, prompt injection against any LLM-based component, RBAC on dashboard

5. Operations and SLOs

Item	Target
Gateway availability	99.9%
Inline decision latency (p95)	Under 10 ms
Alert latency from event to dashboard	Under 5 s
Detector outage policy	Sensitive actions fail closed (need approval); low-risk actions fail open and are logged
Monitoring	OpenTelemetry traces, Prometheus metrics, Grafana dashboards, alert on queue lag and error rate
Backups	Daily event-store backup; manifest repo is the source of truth

6. Incident runbooks (short form)

Situation	Steps
Alert flood / false-positive spike	1) Switch the noisy rule to shadow mode. 2) Review top contributing signal. 3) Tune threshold or baseline. 4) Re-enable after replay test passes.
Detector or queue down	1) Confirm fail-closed policy is active for sensitive actions. 2) Restart workers; check queue lag. 3) Replay missed events once healthy.
Confirmed compromised agent	1) Isolate session. 2) Revoke token and rotate secrets. 3) Export forensic snapshot. 4) Review downstream agents named in the delegation chain. 5) Add scenario to the test suite.
Bad manifest change	1) Roll back to the previous signed version. 2) Audit who approved it. 3) Add a validation test.

7. Definition of done for a release

- All CI gates green, including the detection-quality gate.
- Staging canary healthy for 15 minutes and smoke test passed.
- Manifests signed and versioned; changelog written; rollback command tested.